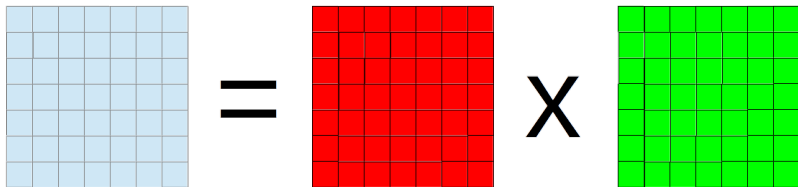


## Unidade - 12

### Matrizes.

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

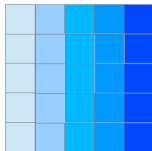

$$C = A \times B$$

$$C_{ij} = \sum_l A_{il} \times B_{lj}$$

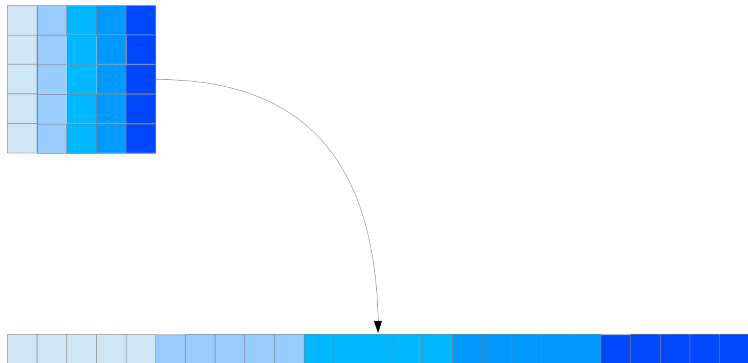
$$C_{ij} = \sum_l A_{il} \times B_{lj}$$

```
DO i=1,10
  DO j=1,10
    C(i,j)=0
    DO l=1,10
      C(i,j)=C(i,j)+A(i,l)*B(l,j)
    END DO
  END DO
END DO
```

# Como o Computador guarda uma matriz?



# Como o Computador guarda uma matriz?



```
C=0.0D0
DO j=1,10
  DO i=1,10
    DO l=1,10
      C(i,j)=C(i,j)+A(i,l)*B(l,j)
    END DO
  END DO
END DO
```

```
C=0.0D0
DO j=1,10
  DO l=1,10
    DO i=1,10
      C(i,j)=C(i,j)+A(i,l)*B(l,j)
    END DO
  END DO
END DO
```

```
C=0.0D0
DO j=1,10
  DO l=1,10
    tmp=B(l,j)
    DO i=1,10
      C(i,j)=C(i,j)+A(i,l)*tmp
    END DO
  END DO
END DO
```

- LAPACK: Linear Algebra PACKage
- BLAS: Basic Linear Algebra Subprograms

- LAPACK: Linear Algebra PACKage
- BLAS: Basic Linear Algebra Subprograms

Acesse

<https://www.netlib.org/lapack/>  
e baixe a última versão.

- LAPACK: Linear Algebra PACKage
- BLAS: Basic Linear Algebra Subprograms

Acesse

<https://www.netlib.org/lapack/>  
e baixe a última versão.

Vamos Compilar!

# Copilando BLAS & LAPACK

- Baixe o pacote “lapack-x.xx.x.tar.gz”
- Descompacte com “tar -xvzf lapack-x.xx.x.tar.gz”
- Entre na pasta descompactada
- Faça uma cópia do arquivo “make.inc.example” com o nome “make.inc”
- Entre em “BLAS/SRC”
- Entre com o comando “ulimit -s unlimited”
- Entre o comando “make”
- Isso compilará a biblioteca BLAS
- Deverá gerar o arquivo “librefblas.a” na pasta principal
- Se tudo der ok, volte duas pastas: “cd ../..”
- Entre com o comando “ulimit -s unlimited”
- Entre o comando “make”
- Isso compilará a biblioteca LAPACK (demora bastante)
- Deverá gerar o arquivo “liblapack.a” na pasta principal
- Se não for exibida nenhuma mensagem de erro, inclusive depois dos testes, as bibliotecas estão prontas para uso.

# Utilizando a biblioteca BLAS

→ Use a subrotina “dgemm”.

- Use a subrotina “dgemm”.
- Abra o código fonte e verifique o preâmbulo para entender:

- Use a subrotina “dgemm”.
- Abra o código fonte e verifique o preâmbulo para entender:
  - O seu uso;

- Use a subrotina “dgemm”.
- Abra o código fonte e verifique o preâmbulo para entender:
  - O seu uso;
  - Seus argumentos.

- Use a subrotina “dgemm”.
- Abra o código fonte e verifique o preâmbulo para entender:
  - O seu uso;
  - Seus argumentos.

Mão à obra!

DGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDB)

DGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDB)

## Propósito

- Queremos  $C = A \times B$

- DGEMM realiza

$$C = \alpha \times \text{op}(A) \times \text{op}(B) + \beta \times C$$

- 

- - 
  -

DGEMM(**TRANSA**,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDB)

## Propósito

- Queremos  $C = A \times B$
- DGEMM realiza

$$C = \alpha \times op(A) \times op(B) + \beta \times C$$

## Argumentos

- TRANSA
  - SE TRANSA='N'  $\rightarrow op(A) = A$
  - SE TRANSA='T' ou 'C'  $\rightarrow op(A) = A^T$
  - Usando TRANSA='N'  $\rightarrow C = \alpha \times A \times op(B) + \beta \times C$

DGEMM(TRANSA, **TRANSB**, M, N, K, ALPHA, A, LDA, B, LDB, BETA, C, LDC)

## Propósito

- Queremos  $C = A \times B$
- DGEMM realiza

$$C = \alpha \times op(A) \times op(B) + \beta \times C$$

## Argumentos

- TRANSB
  - SE TRANSB='N'  $\rightarrow op(B) = B$
  - SE TRANSB='T' ou 'C'  $\rightarrow op(B) = B^T$
  - Usando TRANSB='N'  $\rightarrow C = \alpha \times A \times B + \beta \times C$

DGEMM(TRANSA,TRANSB,**M,N,K**,ALPHA,A,LDA,B,LDB,BETA,C,LDC)

## Propósito

- Queremos  $C = A \times B$
- DGEMM realiza
$$C = \alpha \times op(A) \times op(B) + \beta \times C$$

## Argumentos

- $M \rightarrow$  número de linhas de  $op(A)$  e de  $C$ ;
- $N \rightarrow$  número de colunas de  $op(B)$  e de  $C$ ;
- $K \rightarrow$  núm. de col. de  $op(A)$  e linhas de  $op(B)$ .

DGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDO

## Propósito

- Queremos  $C = A \times B$

- DGEMM realiza

$$C = \alpha \times \text{op}(A) \times \text{op}(B) + \beta \times C$$

## Argumentos

- ALPHA=1.0D0  $\rightarrow C = A \times B + \beta \times C$ ;
- BETA=0.0D0  $\rightarrow C = A \times B$ ;

DGEMM(TRANSA,TRANSB,M,N,K,ALPHA,A,LDA,B,LDB,BETA,C,LDC)

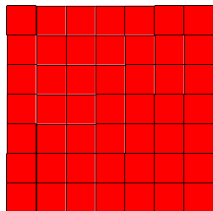
## Propósito

- Queremos  $C = A \times B$
- DGEMM realiza

$$C = \alpha \times \text{op}(A) \times \text{op}(B) + \beta \times C$$

## Argumentos

- LDA  $\rightarrow$  dimensão líder de A;
- LDB  $\rightarrow$  dimensão líder de B;
- LDC  $\rightarrow$  dimensão líder de C.



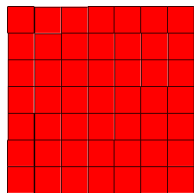
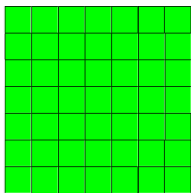
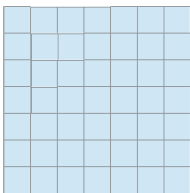
\*



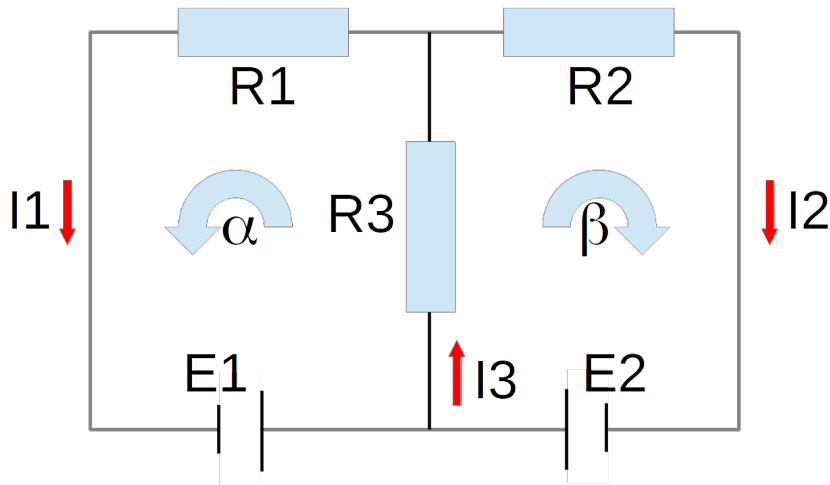
=



$$A * x = b$$

 $X$  $=$ 

$$AX=B$$



$$R_1 I_1 + 0 I_2 + R_3 I_3 = E_1$$

$$0 I_1 + R_2 I_2 + R_3 I_3 = E_2$$

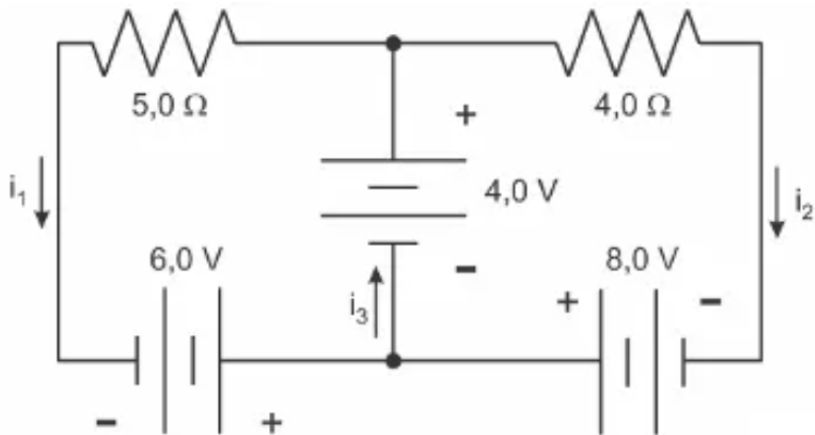
$$I_1 + I_2 - I_3 = 0$$

$$\begin{bmatrix} R1 & 0 & R3 \\ 0 & R2 & R3 \\ 1 & 1 & -1 \end{bmatrix} * \begin{bmatrix} I1 \\ I2 \\ I3 \end{bmatrix} = \begin{bmatrix} E1 \\ E2 \\ 0 \end{bmatrix}$$

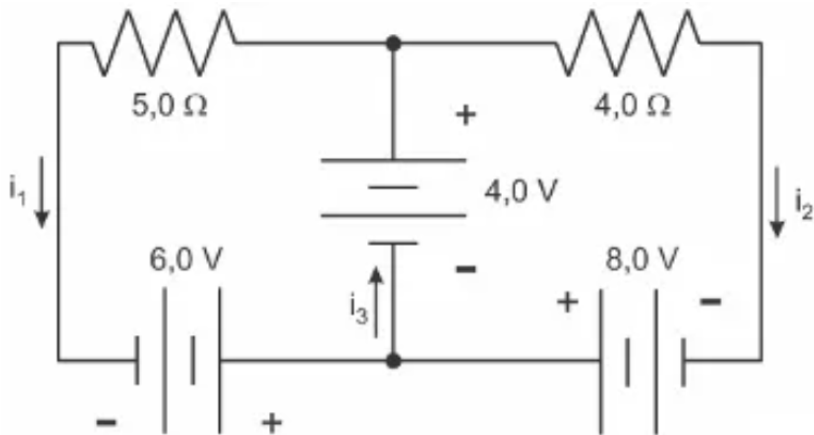
$$\begin{bmatrix} R1 & 0 & R3 \\ 0 & R2 & R3 \\ 1 & 1 & -1 \end{bmatrix} * \begin{bmatrix} I1 \\ I2 \\ I3 \end{bmatrix} = \begin{bmatrix} E1 \\ E2 \\ 0 \end{bmatrix}$$

LAPACK  $\rightarrow$  DGESV

## Exemplo



## Exemplo



Resposta  $\rightarrow i_1 = 2\text{ A}$ ,  $i_2 = 3\text{ A}$ ,  $i_3 = 5\text{ A}$

$$Ax = \lambda x$$

$$Ax = \lambda x$$

- $A \rightarrow$  matriz quadrada;
- $x \rightarrow$  vetor coluna;
- $\lambda \rightarrow$  autovalor.

$$(A - \lambda I)x = 0$$

$$(A - \lambda I)x = 0$$

$$|A - \lambda I| = 0$$

```
SUBROUTINE DSYEV(JOBZ,UPLO,N,A,LDA,W,WORK,LWORK,INFO)
```

SUBROUTINE DSYEV(JOBZ,UPLO,N,A,LDA,W,WORK,LWORK,INFO)

LWORK é o tamanho de WORK.

SUBROUTINE DSYEV(JOBZ,UPLO,N,A,LDA,W,WORK,LWORK,INFO)

LWORK é o tamanho de WORK.

Primeira chamada

- Primeiro alocamos WORK com um tamanho qualquer ...
- ...e chamamos a subrotina com LWORK= - 1
- ...DSYEV nos dará o valor correto de LWORK.

SUBROUTINE DSYEV(JOBZ,UPLO,N,A,LDA,W,WORK,LWORK,INFO)

LWORK é o tamanho de WORK.

Primeira chamada

- Primeiro alocamos WORK com um tamanho qualquer ...
- ...e chamamos a subrotina com LWORK= - 1
- ...DSYEV nos dará o valor correto de LWORK.

Segunda chamada

- Realocamos WORK com o tamanho correto...
- ...e diagonalizamos a matriz